```
      ],
      "score_mode": "avg",
      "boost_mode": "sum"
    }
  }
}
```

The `functions` object in this query expects multiple functions (such as `weight` and `field_value_factor`), which combine to produce a unified score. The `weight` field (the weighting function) expects a positive integer, which is used in further calculations. The original score of fetching a TV using a `term` query is complemented by the following:

- If the brand is LG, increase the score by a weight of 3.
- If user ratings are in the range 4.5 to 5, use the `user_rating` field's value and square it by a factor of 5.

As more functions match, the score's final value increases. Thus the document may appear at the top of the list.

Function scoring modes

Did you notice the `score_mode` and `boost_mode` fields at the end of the script? These two attributes of the `function_score` query allow us to achieve a combined score from the original query and the score emitted by a single or many functions.

By default, the scores produced by these functions are all multiplied to get to a single, final score. However, we can change that behavior by setting the `score_mode` attribute in the `function_score` query. The `score_mode` attribute defines how individual scores are computed. For example, if the `score_mode` of a query is set to `sum`, the scores emitted by the individual functions are all summed up. The `score_mode` attribute can be any mode, such as `multiply` (default), `sum`, `avg`, `max`, `min`, or `first`.

The score from these functions will then be added to (or multiplied by, or averaged with, etc.) the original score of the query (in the example, the `term` query that finds TVs) from the document based on the `boost_mode` parameter. The `boost_mode` parameter can be `multiply` (default), `min`, `max`, `replace`, `avg`, or `sum`. To learn more about the modes and the mechanics involved in function scoring, consult Elasticsearch's official documentation.

That's a wrap! This chapter introduced compound queries that are advanced, useful, and practical. The `bool` query is the Swiss Army knife of all queries, and it helps build complex search queries. In the next chapter, we look at other advanced searches, such as geospatial searches and join queries.

Summary

- Compound queries combine leaf queries to create advanced queries that satisfy multiple search criteria.